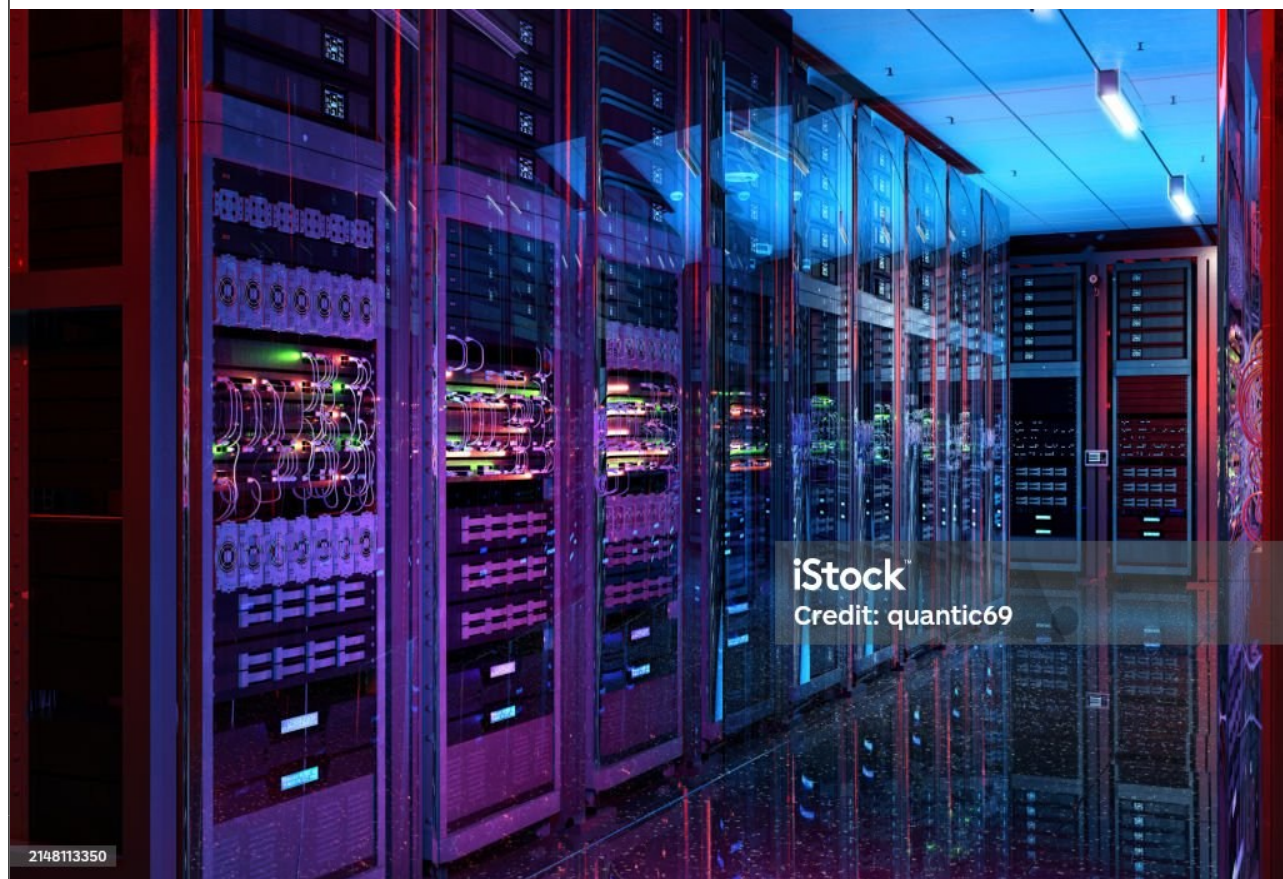




**NAME:** KUTLOANO  
**SURNAME:** MAVOVANA  
**STUDENT NUMBER:** 228023483  
**MODULE CODE:** WRDV301



# Table of Contents

## Contents

|                        |    |
|------------------------|----|
| Table of Contents..... | 1  |
| Rubric .....           | 1  |
| Class Submissions..... | 3  |
| QUIZ.....              | 3  |
| CLASS ACTIVITIES ..... | 4  |
| 3NF .....              | 4  |
| QUERIES.....           | 5  |
| DB DESIGN.....         | 6  |
| ASSIGNMENTS.....       | 7  |
| ASSIGNMENT 1 .....     | 7  |
| Draft version.....     | 7  |
| Final version .....    | 10 |
| Re-Submission .....    | 13 |
| ASSIGNMENT 2 .....     | 17 |
| Draft version.....     | 17 |
| Final version .....    | 20 |
| ASSIGNMENT 3 .....     | 22 |
| Draft version.....     | 22 |
| Final version .....    | 24 |
| ASSIGNMENT 4 .....     | 26 |
| Draft version.....     | 26 |
| Final version .....    | 27 |
| ASSIGNMENT 5 .....     | 29 |
| Draft version.....     | 29 |
| Final version .....    | 31 |
| REFLECTION .....       | 32 |

## Rubric

Class Activities (part of the 60% component of the portfolio mark)

| CLASS ACTIVITY          | YOUR MARK | MAX MARK | CLASS AVG | COMMENT                         |
|-------------------------|-----------|----------|-----------|---------------------------------|
| 01 – 13 FEB*            | y         | n/a      | n/a       | Attendance – nothing to include |
| 02 – 20 FEB*            | y         | n/a      | n/a       | Attendance – nothing to include |
| 03 – 27 FEB*            | y         | n/a      | n/a       | Attendance – nothing to include |
| 03 – 27 FEB (QUIZ)      | 6         | 6        | 4/6       | Include in portfolio            |
| 05 – 13 MAR (3NF)       | 10        | 10       | 9/10      | Include in portfolio            |
| 06 – 27 MAR (QUERIES)   | 10        | 10       | 9/10      | Include in portfolio            |
| 07 – 10 APR (DB DESIGN) | 10        | 20       | 12/20     | Include in portfolio            |

88%

Preparation / assignments (part of the 60% component of the portfolio mark)

| ASSIGNMENT  | YOUR MARK | MAX MARK | CLASS AVG | Max of one:<br><i>Either RESUBMIT or FIRST SUBMIT</i><br>* Indicate with  where relevant |              |
|-------------|-----------|----------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
|             |           |          |           | RESUBMIT                                                                                                                                                                     | FIRST SUBMIT |
| 01 – 3 MAR  | 63.5      | 70       | 56/70     | ✓*                                                                                                                                                                           | Mark         |
| 02 – 17 MAR | 31        | 36       | 29/36     | ✓*                                                                                                                                                                           | 66/70        |
| 03 – 31 MAR | 31        | 31       | 29/31     |                                                                                                                                                                              |              |
| 04 – 21 APR | 31        | 35       | 30/35     |                                                                                                                                                                              |              |
| 05 – 12 MAY | 17        | 19       | 15/19     |                                                                                                                                                                              |              |

Semester tests (60% of final mark)

|        | YOUR MARK | MAX MARK | CLASS AVG |
|--------|-----------|----------|-----------|
| 18 MAR | 36        | 60       | 36/60     |
|        |           |          |           |

Portfolio mark (40% of final mark)

| 60% Component | 40% Component |
|---------------|---------------|
| 87            |               |

| Overview Component        | Does the criterion meet the expectations?   |    |                                                 |           |                                         |     |                                                                      |         |       |
|---------------------------|---------------------------------------------|----|-------------------------------------------------|-----------|-----------------------------------------|-----|----------------------------------------------------------------------|---------|-------|
|                           | 0                                           | No | 1                                               | Partially | 2                                       | Yes | 3                                                                    | Exceeds | Score |
| 1. Submitted artefacts    | Required sections or artefacts not included |    | Some sections and some artefacts included       |           | All sections and all artefacts included |     | All sections and all artefacts appropriately included and documented |         | 3     |
| 2. Evidence of learning   | Little or no evidence of learning           |    | Some evidence of learning, but not sufficient   |           | Adequate evidence of learning           |     | Clear evidence of learning                                           |         | 3     |
| 3. Evidence of reflection | Little or no evidence of reflection         |    | Some evidence of reflection, but not sufficient |           | Adequate evidence of reflection         |     | Clear evidence of reflection                                         |         | 2     |
| 4. Presentation           | Instructions not followed; many errors      |    | Most instructions followed; still some errors   |           | Well organised; instructions followed   |     | Well organised, well structured, all instructions followed           |         | 1     |

65%

# Class Submissions

## QUIZ

### Weak / Strong entities

|                  |                                      |
|------------------|--------------------------------------|
| <b>Status</b>    | Finished                             |
| <b>Started</b>   | Thursday, 27 February 2025, 12:35 PM |
| <b>Completed</b> | Thursday, 27 February 2025, 12:36 PM |
| <b>Duration</b>  | 1 min 44 secs                        |
| <b>Grade</b>     | 6.00 out of 6.00 (100%)              |

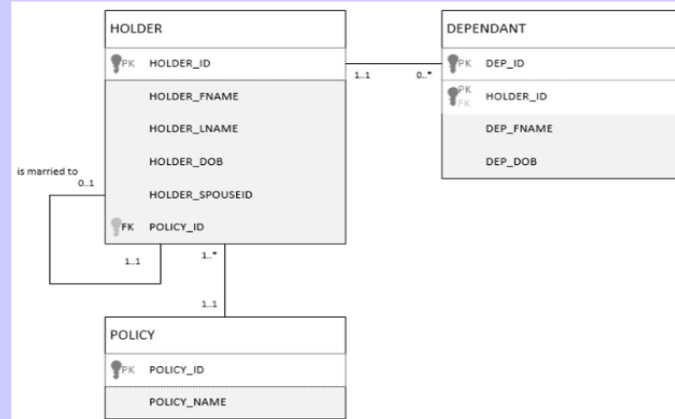
#### Question 1

Complete

Mark 3.00 out of 3.00

Flag question

Consider the following diagram. Is DEPENDANT a weak or strong entity?



Select one:

- ☐ Strong - it is existence dependent and its PK is fully derived from the parent entity
- ☐ Strong - it is existence independent and its PK is not derived from the parent entity
- ☐ Weak - it is existence dependent and its PK is partially derived from the parent entity
- ☐ Weak - it is existence dependent and its PK is not derived from the parent entity

- ☐ Weak - it is existence independent and its PK is partially derived from the parent entity
- ☐ Strong - it is existence independent and its PK is partially derived from the parent entity

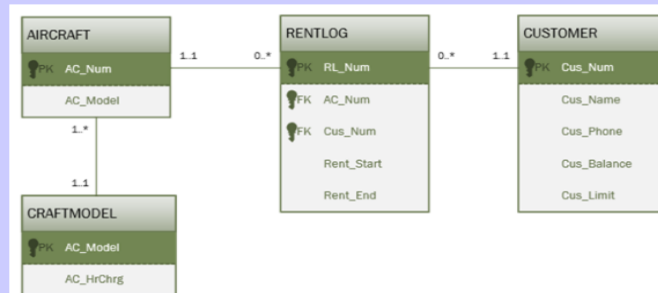
#### Question 2

Complete

Mark 3.00 out of 3.00

Flag question

Consider the following diagram. Is RENTLOG a weak or strong entity?



Select one:

- ☐ Weak - it is existence independent and its PK is partially derived from the parent entity
- ☐ Weak - it is existence dependent and its PK is not derived from the parent entity
- ☐ Strong - it is existence independent and its PK is not derived from the parent entity
- ☐ Strong - it is existence dependent and its PK is fully derived from the parent entity
- ☐ Weak - it is existence dependent and its natural PK is partially derived from the parent entity
- ☐ Strong - it is existence dependent and its PK is not derived from the parent entity

## CLASS ACTIVITIES

## 3NF

Names : Kutloano Mavovana 228023483  $\frac{10}{10}$   
Sandle Rasmeni 226158187  
Thulego Potlone 228086388

Class Activity : Normalization  
Hardware Store

1NF :

SalesPeople (spNum, spName, spCompere, hireYear, deptNum,  
deptName, prodNum, prodName, prodPrice, qtySold)

3NF :

SalesPeople (spNum, deptNum, prodNum, qtySold) ✓✓

SalesPerson (spNum, spName, spCompere, hireYear) ✓

Department (deptNum, deptName) ✓

Product (prodNum, prodName, prodPrice) ✓

## QUERIES

### MURDER MYSTERY-THUTEGO POTSAANE-KUTLOANO MAVOVANA-SANDILE RASIMENI

1.

a.

```

select p.name
from get_fit_now_member as fnm join person as p on p.id=fnm.person_id
join get_fit_now_check_in as fnc on fnc.membership_id=fnm.id
join interview as i on p.id=i.person_id
join crime_scene_report as csr
join facebook_event_checkin as fec on p.id=fec.person_id
join drivers_license as dl on p.license_id=dl.id
join income as inc on p.ssn=inc.ssn
where fec.date=csr.date
and csr.city = 'SQL City' and csr.type='murder' and csr.date=20180115

```

10/10

b.

| name          |
|---------------|
| Jeremy Bowers |

Did you find the killer?

```

1 INSERT INTO solution VALUES (1, 'Jeremy Bowers');
2
3 SELECT value FROM solution;

```

RUN ↴

RESET

value

Congrats, you found the murderer! But wait, there's more... If you think you're an interview transcript of the murderer to find the real villain behind this crime. If you have good SQL skills, try to complete this final step with no more than 2 queries. Use this s

2.

```

select p.name, count(fec.person_id) as attendance
from person as p join drivers_license as dl on p.license_id=dl.id
join facebook_event_checkin as fec on p.id=fec.person_id
join income as i on p.ssn=i.ssn
where dl.height in (65,67)
and dl.hair_color='red'
and dl.car_model='Model S'
and dl.car_make='Tesla'
and dl.gender='female'
and fec.event_name='SQL Symphony Concert'
and fec.date Like '201712%'
group by p.name
having count(fec.person_id)=3

```

| name             | attendance |
|------------------|------------|
| Miranda Priestly | 3          |

```
1 INSERT INTO solution VALUES (1, 'Miranda Priestly');
```

```
SELECT value FROM solution;
```

RESET

value

Congrats, you found the brains behind the murder! Everyone in SQL City hails you as the greatest SQL detective. Time to break out the champagne!

## DB DESIGN

## WRDV301 Class Activity

| Name     | Surname  | Student Number |
|----------|----------|----------------|
| Kutloano | Mavovana | 228023483      |
| Thutego  | Potsane  | 228086388      |
| Sandile  | Rasimeni | 226158187      |

## 1. RELATIONAL SCHEMAS:

- Supplier(suppID, suppFName, suppLName, suppCell\_Num)
- BusinessCustomer(custID, bussName, bussPhone\_Num, streetNumber, streetName, town, city)
- IndividualCustomer(custID, indivFName, indivLName, streetNumber, streetName, town, city)
- SalePerson(spID, spFName, spLName, cell\_Num, email)
- Order(orderNum, quantity, orderDate, itemNum, custID, VAT\_Percentage, discPercentage, deliveryID, spID) *Delivery address?*
- Item(itemNum, itemDescription, itemPrice, suppID) *implies an item is supplied by a single supplier*
- Invoice(invoiceNum, invoiceDate, deliveryID, orderNum, itemNum, subtotal, TAX, deliveryDate)
- Delivery(deliveryID, deliveryDate, deliveryFee, city, streetName, postalCode, suburb, province, cellNum)

*OrderLine?**InvoiceLine?*

| ITEM              | MARK | MAX |
|-------------------|------|-----|
| Customer          | 2    | 2   |
| Item              | 2    | 2   |
| Order             | 1    | 3   |
| OrderLine         | 0    | 2   |
| Invoice           | 1    | 3   |
| InvoiceLine       | 0    | 2   |
| Sales Person      | 2    | 2   |
| PK/FK             | 0    | 2   |
| Atomic Attributes | 2    | 2   |
|                   | 10   | 20  |

# ASSIGNMENTS

## ASSIGNMENT 1

Draft version

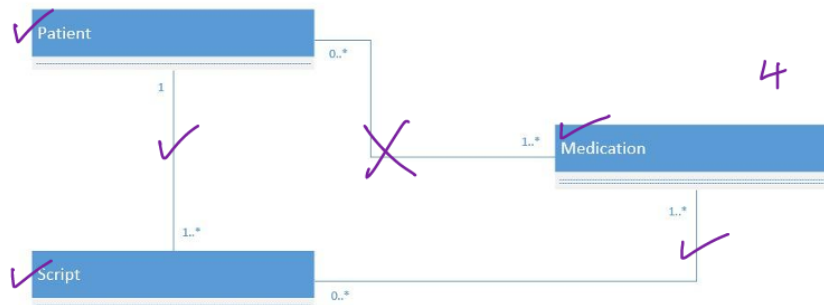
### Assignment 1-Draft

40  
10

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Rasimeni, Sandile  |
| Surname, Name | Mavovana, Kutloano |

#### Chapter 2

- 1.
- a) A patient must receive at least one medication per day  
 A patient can receive scripts prescribed while in hospital  
 Script can have multiple medications on it  
 Script is given to one patient only  
 Medication can be prescribed on one or more scripts  
 Medication can be prescribed to one or more patients *implied by above*
- b) The ERD below:



#### Chapter 3

- 1.
- a) For each of the tables

| Table | Primary Key | Candidate Key                                                | Foreign Key(s)         |
|-------|-------------|--------------------------------------------------------------|------------------------|
| BASE  | BASE_CODE ✓ | {BASE_PROV, BASE_PROV, BASE_AREA_CODE}<br><i>not minimal</i> | ✓                      |
| TRUCK | TRUCK_NUM ✓ | {TRUCK_PURCH_DAT, TRUCK_SERIAL_NUM}<br><i>not minimal</i>    | BASE_CODE, TYPE_CODE ✓ |
| TYPE  | TYPE_CODE ✓ | TYPE_CODE ✓                                                  | ✓                      |

- b) Do all tables exhibit entity integrity?

| Table | Yes / No                            | Explanation                                                    |
|-------|-------------------------------------|----------------------------------------------------------------|
| BASE  | <input checked="" type="checkbox"/> | BASE_CODE is a unique primary key, containing no NULL values ✓ |
| TRUCK | <input checked="" type="checkbox"/> | TRUCK_NUM is a unique primary key, containing no NULL values ✓ |
| TYPE  | <input checked="" type="checkbox"/> | TYPE_CODE is a unique primary key, containing no NULL values ✓ |

- c) Do all tables exhibit referential integrity?

| Table | Yes / No                            | Explanation                                                                                                                                                                      |
|-------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BASE  | <input type="checkbox"/>            | No FK containing a value that refers to existing valid entry in another relation. ✓                                                                                              |
| TRUCK | <input checked="" type="checkbox"/> | BASE_CODE and TYPE_CODE contains values that refers to existing valid entry in another relation.<br>BASE_CODE has a NULL value since its not the part of the PK in this table. ✓ |
| TYPE  | <input type="checkbox"/>            | No FK containing a value that refers to existing valid entry in another relation. ✓                                                                                              |

## Chapter 4

3.

a)  $\pi_{\text{Vend\_Prod}}(\text{Vending}) \cup \pi_{\text{Kiosk\_Prod}}(\text{Kiosk})$ 

b)

| Vend_Prod | Vend_Price |
|-----------|------------|
| Mentos    | 9.5        |
| Mentos    | 7.5        |
| Monster   | 25         |
| PRIME     | 39         |
| BarOne    | 16         |

c)

2 | Page

| Vend_Prod | Vend_Price |
|-----------|------------|
| BarOne    | 16         |

d)

| Kios_Prod | Kios_Price |
|-----------|------------|
| Mentos    | 7.5        |
| PRIME     | 39         |

4.

a)  $\pi_{\text{AuthName}}(\sigma_{\text{PubDate} > '01/07/2022'}(\text{BOOK}))$ b)  $\pi_{\text{ISBN}}(\text{BOOK}) \cap \pi_{\text{ISBN}}(\sigma_{\text{Quantity} > 0}(\text{STOCK}))$ 

Why?

## Chapter 5

5.

a)

## Unary Relationship (Degree 1)

- (6) • An **EMPLOYEE** manages other **EMPLOYEES**. ✓

## Binary Relationship (Degree 2)

- An **EMPLOYEE** can make many bookings, but **BOOKINGS** are done by only one **EMPLOYEE**. } NO ✓
- A **TRAVELAGENCY** must have at least one **EMPLOYEE**, but **EMPLOYEES** must belong to one and only **TRAVELAGENCY**. ✓
- A **FLIGHT** must consist of many **BOOKINGS**, but **BOOKINGS** must only be done for one **FLIGHT** only. } No
- A **HOTEL** can have many **BOOKINGS**, but **BOOKINGS** must only be done for one **HOTEL** only. } No
- A **CUSTOMER** must make **BOOKINGS**, but **BOOKINGS** are done by one **CUSTOMER** per time. } No
- An **EVENT** can have **BOOKINGS**, but **BOOKINGS** are done for only one **EVENT**. ✓

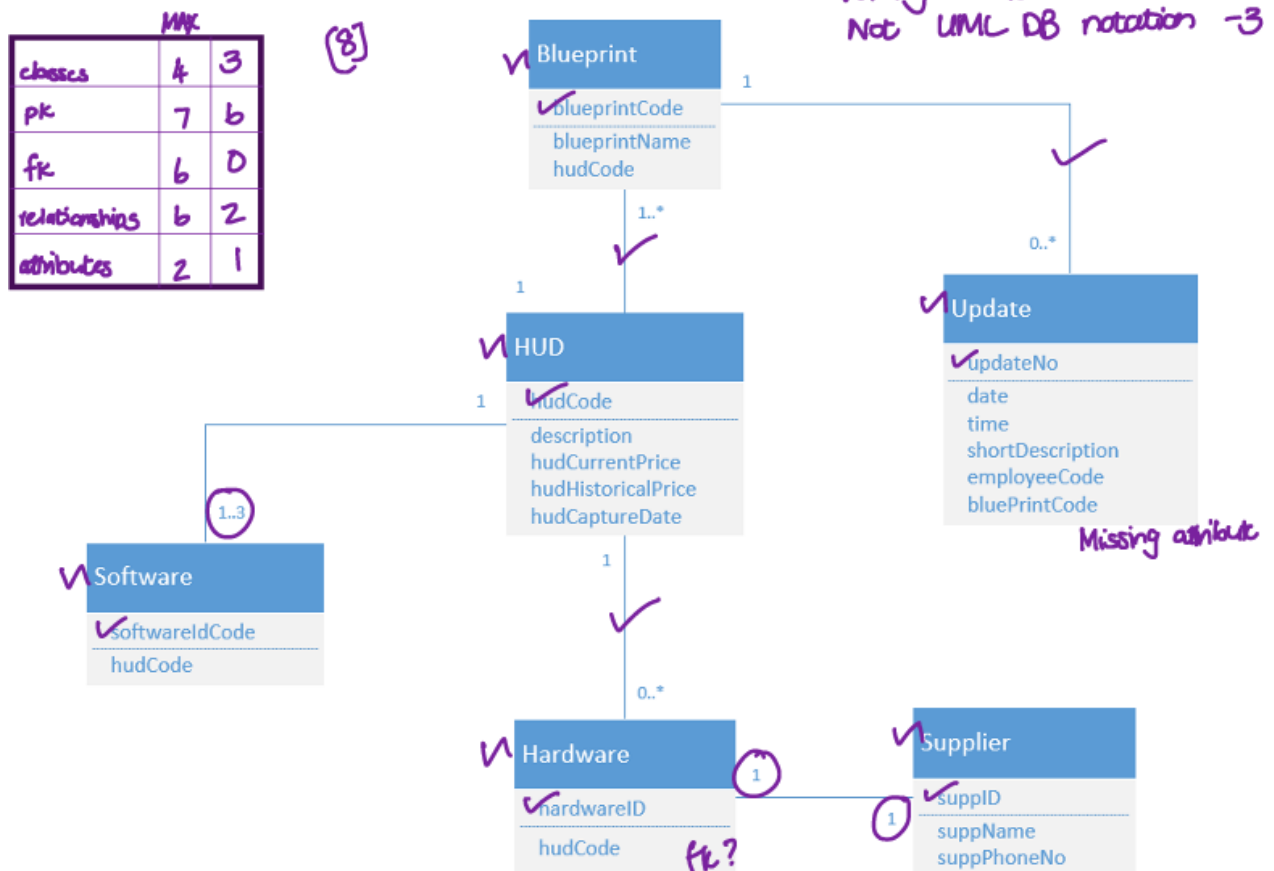
## Ternary Relationship (Degree 3) Higher order

- **CUSTOMER** and **EMPLOYEE** make a **BOOKING** to reserve at the **HOTEL**. } No 4
- **CUSTOMER** and **EMPLOYEE** make a **BOOKING** to attend an **EVENT**. } No
- **CUSTOMER** and **EMPLOYEE** make a **BOOKING** to reserve a sit to the **FLIGHT**. } No

b) **BOOKING** is a weak entity, because ✓

- It depends on entities which it has relationship with (existence-dependent). Which are? 2
- It references other entities using **foreign keys** from strong entity to form a **super key** with extra attributes (FKs). ✓

6.



## Final version

## Assignment 1-Draft

 $\frac{60\frac{1}{2}}{10}$ 

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Rasimeni, Sandile  |
| Surname, Name | Mavovana, Kutloano |

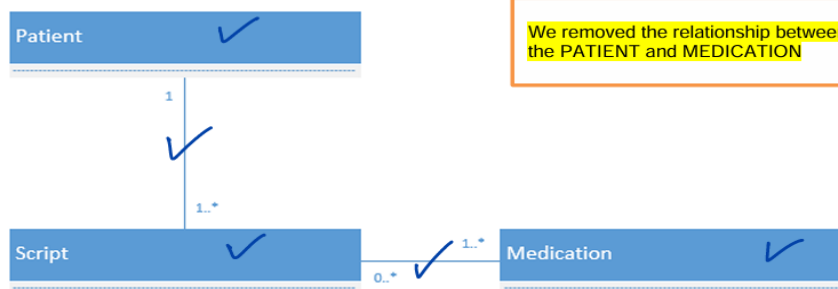
## Chapter 2

- 1.
- A patient must receive at least one medication per day  
A patient can receive scripts prescribed while in hospital  
Script can have multiple medications on it  
Script is given to one patient only  
Medication can be prescribed on one or more scripts
  - The ERD below:

[9]

4

Header?



5

## Chapter 3

- 2.
- For each of the tables

[12½]

| Table | Primary Key | Candidate Key                                       | Foreign Key(s)          |
|-------|-------------|-----------------------------------------------------|-------------------------|
| BASE  | BASE_CODE   | BASE_AREA_CODE<br><i>not ideal, but possible</i>    |                         |
| TRUCK | TRUCK_NUM   | TRUCK_SERIAL_NUM                                    | BASE_CODE,<br>TYPE_CODE |
| TYPE  | TYPE_CODE   | TYPE_CODE<br><i>PK? sorry - missed it in draft.</i> |                         |

4½

- Do all tables exhibit entity integrity?

| Table | Yes / No | Explanation                                                  |
|-------|----------|--------------------------------------------------------------|
| BASE  | Yes      | BASE_CODE is a unique primary key, containing no NULL values |
| TRUCK | Yes      | TRUCK_NUM is a unique primary key, containing no NULL values |

3

|       |     |                                                              |
|-------|-----|--------------------------------------------------------------|
| TRUCK | Yes | TRUCK_NUM is a unique primary key, containing no NULL values |
| TYPE  | Yes | TYPE_CODE is a unique primary key, containing no NULL values |

c) Do all tables exhibit referential integrity?

| Table | Yes / No | Explanation                                                                                                                                                                    |
|-------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BASE  | No       | No FK containing a value that refers to existing valid entry in another relation.                                                                                              |
| TRUCK | Yes      | BASE_CODE and TYPE_CODE contains values that refers to existing valid entry in another relation.<br>BASE_CODE has a NULL value since its not the part of the PK in this table. |
| TYPE  | No       | No FK containing a value that refers to existing valid entry in another relation.                                                                                              |

#### Chapter 4

3.

a)  $\pi_{Vend\_Prod(Vending)} \cup \pi_{Kiosk\_Prod(Kiosk)}$

b)

| Vend_Prod | Vend_Price |
|-----------|------------|
| Mentos    | 9.5        |
| Mentos    | 7.5        |
| Monster   | 25         |
| PRIME     | 39         |
| BarOne    | 16         |

2 | Page

c)

| Vend_Prod | Vend_Price |
|-----------|------------|
| BarOne    | 16         |

d)

d)

| Kiosk_Prod | Kiosk_Price |
|------------|-------------|
| Mentos     | 7.5         |
| PRIME      | 39          |

4.

a)  $\pi_{AuthName} (\sigma_{PubDate > '01/07/2022'}(BOOK))$

b)  $\pi_{ISBN} (\sigma_{Quantity > 0}(STOCK))$

#### Chapter 5

5.

a)

##### Unary Relationship (Degree 1)

- An **EMPLOYEE** manages other **EMPLOYEEES**.

##### Binary Relationship (Degree 2)

- A **TRAVELAGENCY** must have at least one **EMPLOYEE**, but **EMPLOYEEES** must belong to one and only one **TRAVELAGENCY**.

##### Ternary Relationship (Degree 3) – Higher Order

- There exists a ternary relationship between **CUSTOMER**, **HOTEL**, **EVENT**, **FLIGHT**, **BOOKING** and **EMPLOYEE** with the association class being the **EMPLOYEE**.

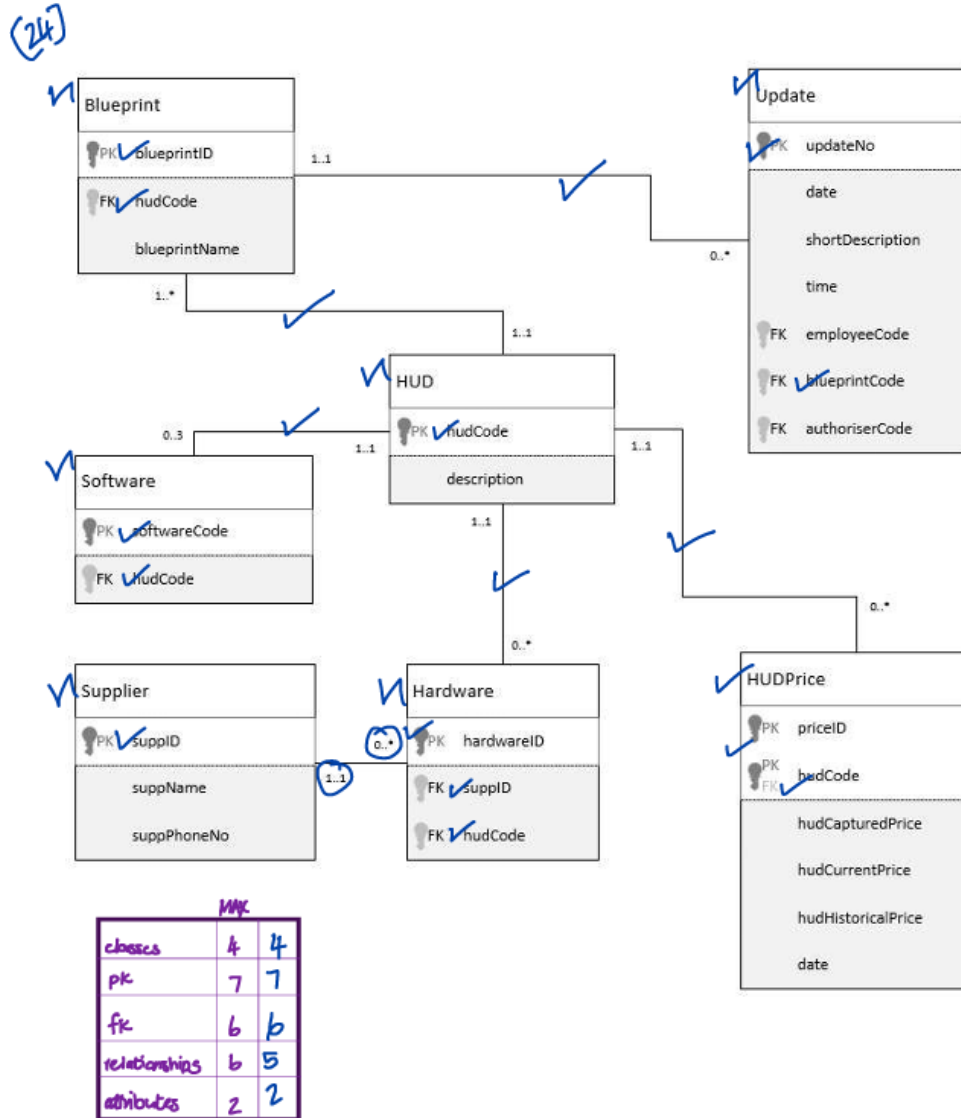
b) **BOOKING** is a weak entity, because

- It depends on the entities which it has relationship with (existence-dependent) which are **FLIGHT**, **HOTEL**, **EVENT**, **EMPLOYEE** and **CUSTOMER**.
- It references other entities using **foreign keys** from strong entity to form a **super key** with extra attributes (FKs).

Assigned pk is not natural pk.

Are fks mandatory?

6. Here we had to change the design of our UML to a database to Notation. There was something wrong with the update table, we fixed that also. We added a new table, but it will be great to just remark the whole design since we made a lot of changes to the original one, we fixed our foreign key, and primary keys.



## Re-Submission

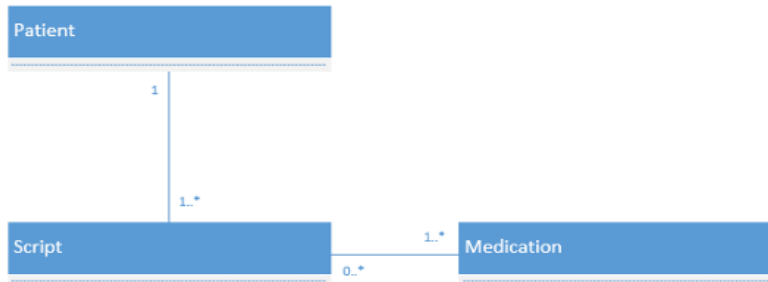
## Assignment 1- Resubmission

66  
70

|               |                    |
|---------------|--------------------|
| Surname, Name | Mavovana, Kutloano |
|---------------|--------------------|

## Chapter 2

- 1.
- A patient must receive at least one medication per day  
A patient can receive scripts prescribed while in hospital  
Script can have multiple medications on it  
Script is given to one patient only  
Medication can be prescribed on one or more scripts
  - The ERD below:



## Chapter 3

- 2.
- For each of the tables

| Table | Primary Key | Candidate Key                  | Foreign Key(s)          |
|-------|-------------|--------------------------------|-------------------------|
| BASE  | BASE_CODE   | BASE_CODE<br><i>already pk</i> | N/A                     |
| TRUCK | TRUCK_NUM   | TRUCK_SERIAL_NUM               | BASE_CODE,<br>TYPE_CODE |
| TYPE  | TYPE_CODE   | TYPE_CODE<br><i>already pk</i> | N/A                     |

- Do all tables exhibit entity integrity?

| Table | Yes / No | Explanation                                                  |
|-------|----------|--------------------------------------------------------------|
| BASE  | Yes      | BASE_CODE is a unique primary key, containing no NULL values |
| TRUCK | Yes      | TRUCK_NUM is a unique primary key, containing no NULL values |
| TYPE  | Yes      | TYPE_CODE is a unique primary key, containing no NULL values |

- Do all tables exhibit referential integrity?

| Table | Yes / No | Explanation                                                                                                                                                                    |
|-------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BASE  | No       | No FK containing a value that refers to existing valid entry in another relation.                                                                                              |
| TRUCK | Yes      | BASE_CODE and TYPE_CODE contains values that refers to existing valid entry in another relation.<br>BASE_CODE has a NULL value since its not the part of the PK in this table. |
| TYPE  | No       | No FK containing a value that refers to existing valid entry in another relation.                                                                                              |

## Chapter 4

- 3.
- $\pi_{Vend\_Prod}(Vending) \cup \pi_{Kiosk\_Prod}(Kiosk)$
  -

| Vend_Prod | Vend_Price |
|-----------|------------|
| Mentos    | 9.5        |
| Mentos    | 7.5        |
| Monster   | 25         |
| PRIME     | 39         |
| BarOne    | 16         |

c)

| Vend_Prod | Vend_Price |
|-----------|------------|
| BarOne    | 16         |

d)

| Kiosk_Prod | Kiosk_Price |
|------------|-------------|
| Mentos     | 7.5         |
| PRIME      | 39          |

4.

a)  $\pi_{AuthName} (\sigma_{PubDate > '01/07/2022'}(BOOK))$ b)  $\pi_{ISBN} (STOCK)$  need to check3  
0

## Chapter 5

5.

a)

## Unary Relationship (Degree 1)

- An **EMPLOYEE** manages other **EMPLOYEES**. ✓

## Binary Relationship (Degree 2)

- A **TRAVELAGENCY** must have at least one **EMPLOYEE**, but **EMPLOYEES** must belong to one and only one **TRAVELAGENCY**. ✓

## Ternary Relationship (Degree 3) – Higher Order

- There exists a ternary relationship between **CUSTOMER**, **HOTEL**, **EVENT**, **FLIGHT**, **BOOKING**. ✓
- A booking is made by a customer, managed by an employee, and includes a flight (**BOOKING**, **CUSTOMER**, **EMPLOYEE**, and **FLIGHT** entities). ✓
- A booking made by a customer, managed by an Employee, and includes a hotel (**BOOKING**, **CUSTOMER**, **EMPLOYEE**, and **HOTEL** entities.). So the **BOOKING** table links (has a relationship with) **CUSTOMER**, **EMPLOYEE**, **EVENT**, **HOTEL**, and **FLIGHT**. ✓

b) **BOOKING** is a weak entity, because ✓

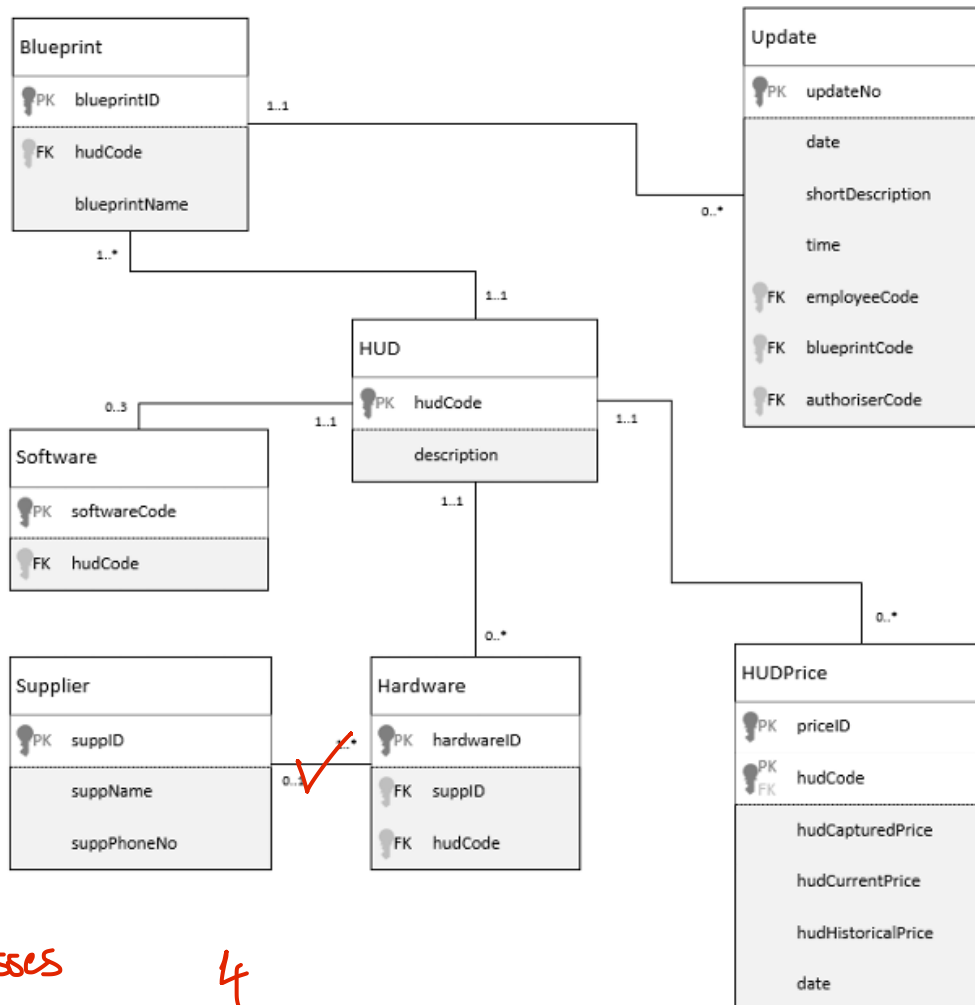
- It depends on the entities which it has relationship with (existence-dependent) which are **FLIGHT**, **HOTEL**, **EVENT**, **EMPLOYEE** and **CUSTOMER**. ✓
- Its Primary key is partially derived from the parent entity. It has a mandatory foreign key the custID, flightID, emplID, hotelID, and eventID. All they are required to identify a booking. ✓

natural

7  
3

6. I have changed the relation between the supplier and hardware (specific cardinalities/ multiplicities).

(25)



Classes 4  
 pk 7  
 fk 6  
 relationships 6  
 attributes 2

## Reflection

**In number 2 a).** We listed BASE\_AREA\_CODE as a candidate key, which is not correct, as the Candidate key must be uniquely identified for each row in the table. So, from the data provided BASE\_AREA\_CODE is not unique across all rows (e.g., multiple bases might share the same area code at some point). Therefore BASE\_AREA\_CODE cannot be a candidate key. Also, I understood that TYPE\_CODE was the PK of the TYPE table, but I wasn't fully clear on whether it was also a candidate key, or what that distinction even meant. Soon after reviewing the feedback and going back to definitions of the two type keys, I realized that PK is always a candidate key as it is minimal by definition.

**In number 4 b).** I acknowledged the misspelling but that is not the case. To this question, no selection is needed, since STOCK table contains only books that are in stock. In other words, every row in the STOCK table represents a book that is available in some quantity at some store. Therefore, there is no need to explicitly filter for *Quantity > 0*. So, omitting the selection, the expression becomes more concise and efficient.

*Books may be out of stock - so you do need to check.*

**In number 5 a).** I acknowledged that there is not valid evidence on how the ternary relationship exists between entities in more detail, on how they all relate and how. Hence, I made some improvements on that, the purpose being that Association Class can exist but not give a clear definition on what is the role of one entity towards the other like saying, Customer makes a Booking, which is managed by Employees, which then includes Hotel and Flights. This will give a clear understanding on how Employees are related to Customers and How Flights and Hotels are related to Customers. So, this clarification provides a more comprehensive understanding of the ternary relationship and association class, enabling more effective database design and management.

**In number 5 b).** As per feedback comment, which says Assigned PK is not natural, hence it's a surrogate PK which was invented to uniquely identify a Booking, since it is related to multiple entities, and depends on. And to answer the Question that are there any Mandatory FKs? Which is a Yes being custID, flightID, emplID, hotelID, and eventID as the Booking must have the identified entities for it to exists, and that makes sense in real i.e., A customer makes a Booking, the Question will be "For What?".

**In number 6.** The relationship between Hardware and Suppliers says that "Suppliers can supply multiple parts, and a supplier may only be on the supplier list if they do supply parts." Which means that Suppliers are listed/ stored in the system is they have Hardware parts that they supplied (a mandatory for a supplier to have one or more hardware(s) supplied in the system). Other side "Each hardware part is supplied by a single supplier" but this is not a case that Hardwares must be supplied by one and only one supplier, since Hardware parts used in the building of the HUDs could either be sourced in-house, which then they don't have any outside suppliers rather the company itself

✓

## ASSIGNMENT 2

Draft version

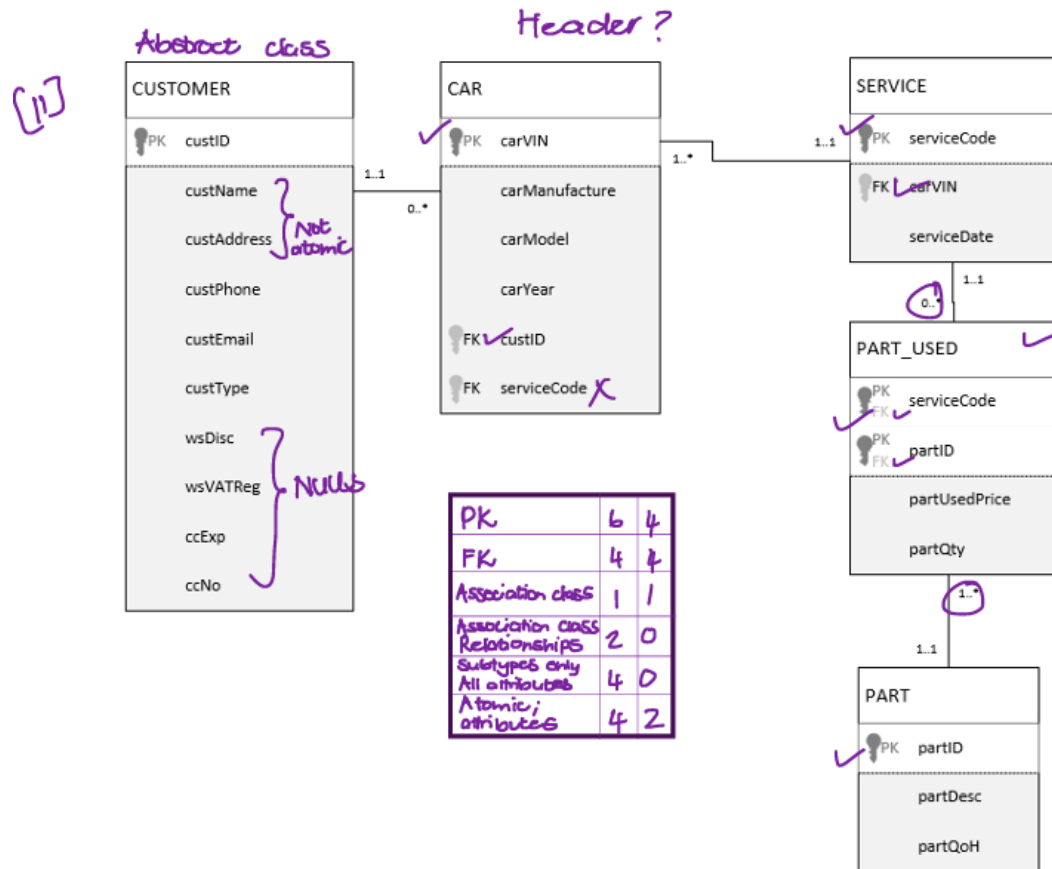
## Assignment 2-Draft

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Rasimeni, Sandile  |
| Surname, Name | Mavovana, Kutloano |



1. See UML Database Notation Design Below:

28/36

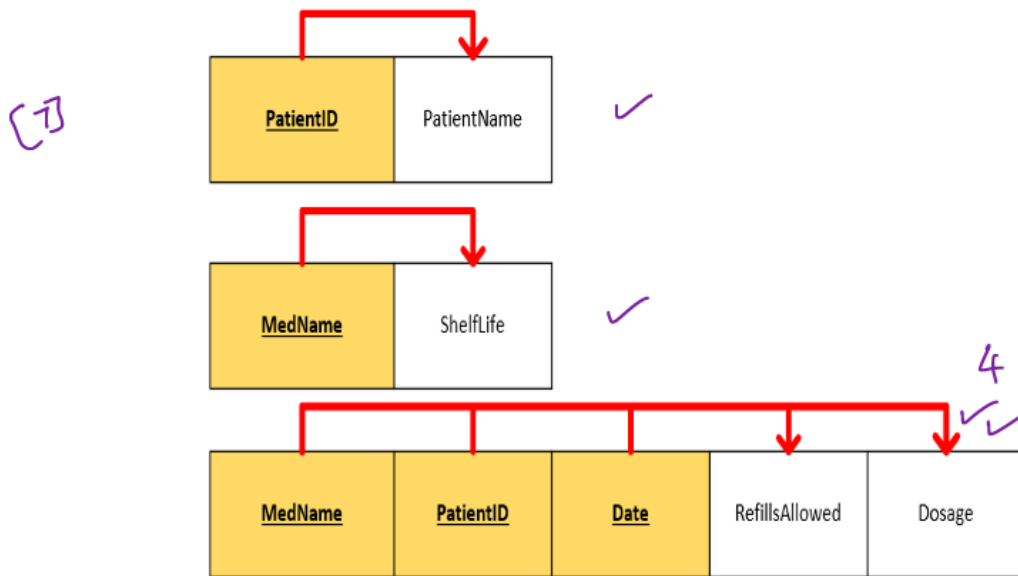


2.

3NF conversion results

3.

a)

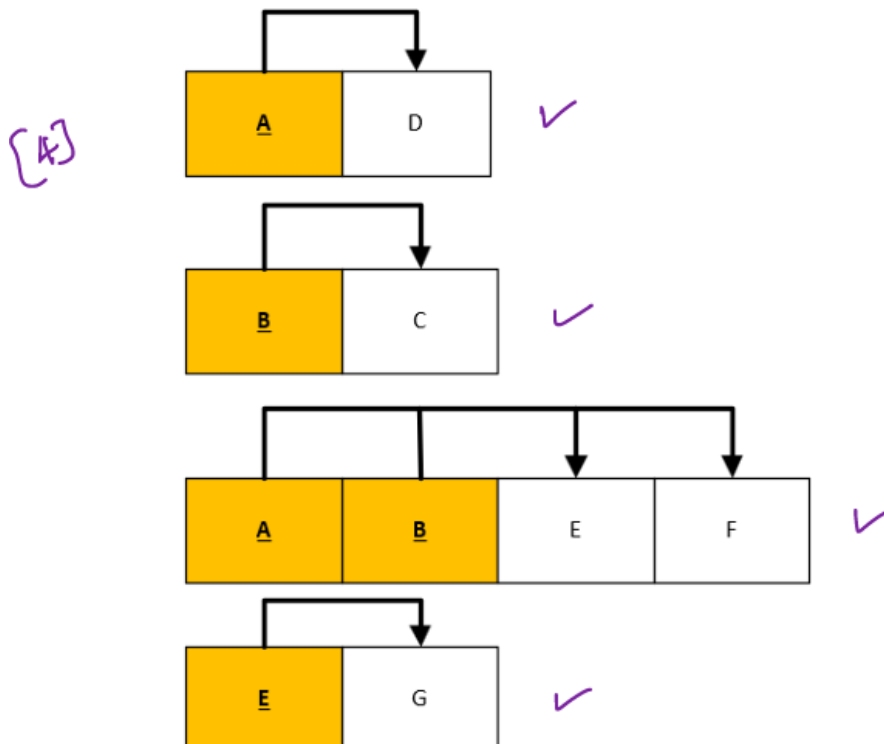


- Q3 a) [6]
- It is already in the 1NF since the PK is identified, which is APP\_NUM. ✓
  - There exists no Partial dependency on the APPOINTMENT entity as there is no attribute that is dependent on only the portion of the primary key. ✓
  - It exhibits transitive dependency, that is one or more attributes are functionally dependent on non-key attributes. i.e., APP\_STATUS can be guessed by only knowing the APP\_DATE, and APP\_TIME such that you can guess that the appointment is closed or not. ✓
  - Therefore, the APPOINTMENT entity is in 2NF. ✓
- 3

b) **DOCTOR** (DOC\_ID[PK], DOC\_LNAME, DOC\_FNAME, DOC\_PHONE) ✓**APPOINTMENT** (APP\_NUM[PK], DOC\_ID[FK], APP\_DATE[FK], APP\_TIME[FK], APP\_PAT\_ID[FK]) ✓**PATIENT** (APP\_PAT\_ID[PK], APP\_PAT\_LNAME, APP\_PAT\_FNAME, APP\_PAT\_PHONE) ✓**STATUS** (APP\_DATE, APP\_TIME, APP\_STATUS) X

Missing attribute ✓

4.



## Final version

Header?

## WRDV301 - ASSIGNMENT 2

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Rasimeni, Sandile  |
| Surname, Name | Mavovana, Kutloano |

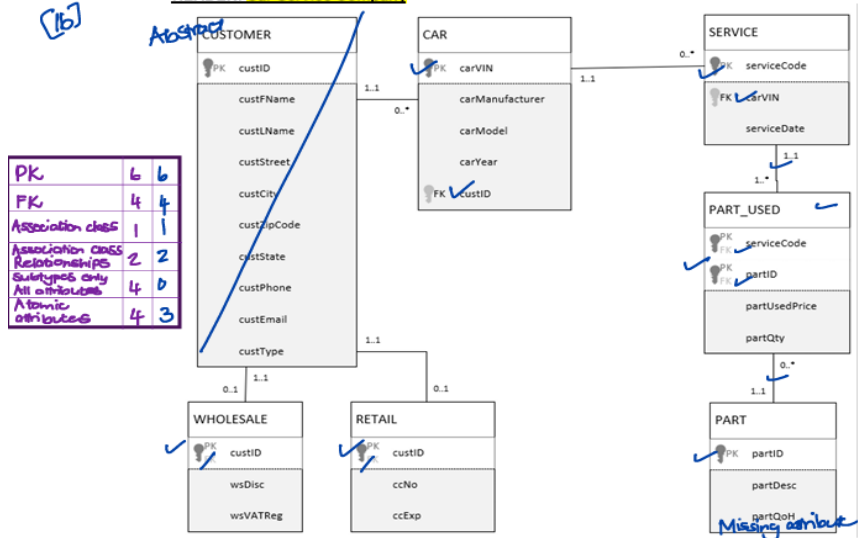
31/36

1.

See UML Database Notation Design Below:

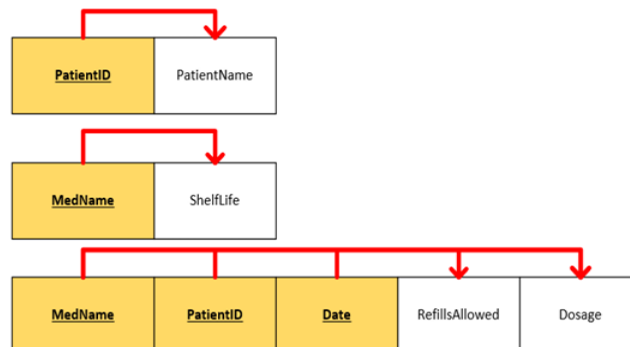
We made changes to the supertype, by implementing the necessary subtypes together with their required attributes and pk and fks. We fixed the association class relationships. We also added the atomic attributes for the customer's name and address. Fixed the relationship between SERVICE AND CAR, we realized you marked it correct, but it was wrong.

HEADER: Car Service Company



## WRDV301 - ASSIGNMENT 2

2. 3NF conversion results



3.

a)

- It is already in the 1NF since the PK is identified, which is APP\_NUM.
- There exists no Partial dependency on the APPOINTMENT entity as there is no attribute that is dependent on only the portion of the primary key.
- It exhibits transitive dependency, that is one or more attributes are functionally dependent on non-key attributes, i.e., APP\_PAT\_FNAME, APP\_PAT\_LNAME, APP\_PAT\_PHONE can be guessed by only knowing the APP\_PAT\_ID.
- Therefore, the APPOINTMENT entity is in 2NF.

b)

DOCTOR (DOC\_ID[PK], DOC\_LNAME, DOC\_FNAME, DOC\_PHONE)

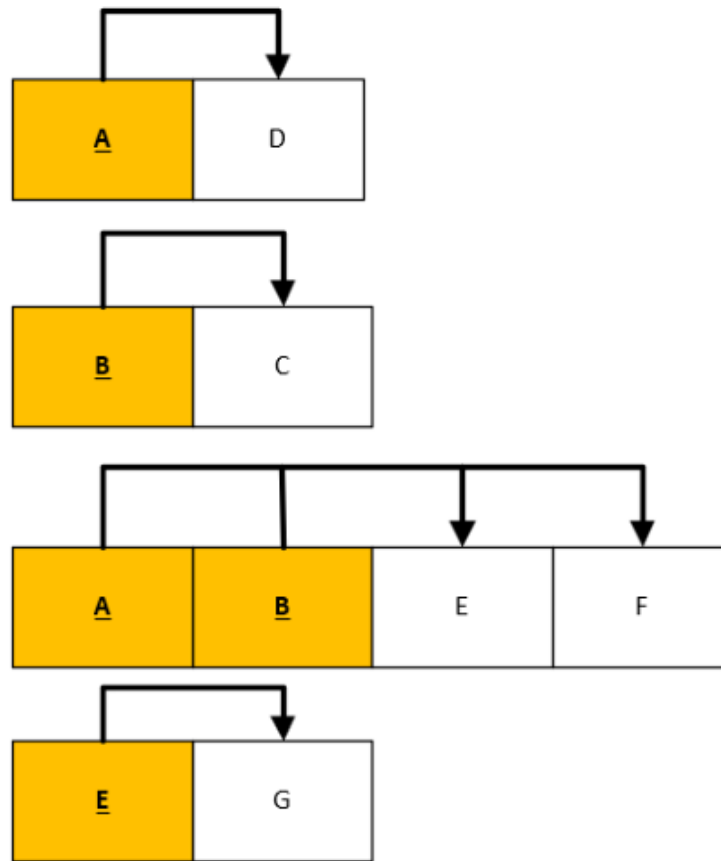
APPOINTMENT (APP\_NUM[PK], DOC\_ID[FK], APP\_DATE, APP\_TIME, APP\_PAT\_ID, APP\_STATUS)

PATIENT (APP\_PAT\_ID[PK], APP\_PAT\_LNAME, APP\_PAT\_FNAME, APP\_PAT\_PHONE)

## WRDV301 - ASSIGNMENT 2

4.

[4]



## ASSIGNMENT 3

Draft version

30  
61

## WRDV301 - ASSIGNMENT 3

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Rasimeni, Sandile  |
| Surname, Name | Mavovana, Kutloano |

1.

a.

`select lLName, lFName, COUNT(Student.sSuperLNo) as NumOfStudents  
from Lecturer join Department on lNo=HoDLNo  
join Student on lNo=sSuperLNo  
group by lLName, lFName  
having COUNT(Student.sSuperLNo)>1`

b.

| Results |          | Messages |               |
|---------|----------|----------|---------------|
|         | ILName   | IFName   | NumOfStudents |
| 1       | Greyling | Jean     | 5             |
| 2       | Mbava    | Willard  | 2             |
| 3       | Venter   | Andre    | 2             |

GOP\potsa (54) | University | 00:00:00 | 3 rows

2.

a.

`SELECT l.lFName as Lecturer_FName, l.lLName as Lecturer_LName, h.lFName  
as HoD_FName, h.lLName as HoD_LName  
FROM Lecturer as l INNER JOIN Department as d on l.lDName = d.dName  
inner join Lecturer AS h on h.lNo=d.HoDLNo  
WHERE l.lNo <> h.lNo  
ORDER BY h.lLName, l.lLName;`

h

b.

|    | Lecturer_FName | Lecturer_LName | HoD_FName | HoD_LName |
|----|----------------|----------------|-----------|-----------|
| 1  | Eric           | Bashman        | Benita    | Barton    |
| 2  | Richard        | Betz           | Benita    | Barton    |
| 3  | Ernst          | Ferg           | Benita    | Barton    |
| 4  | Lynette        | Bamard         | Jean      | Greyling  |
| 5  | Dix            | Dixie          | Jean      | Greyling  |
| 6  | MC             | du Plessis     | Jean      | Greyling  |
| 7  | Ife            | Fashoro        | Jean      | Greyling  |
| 8  | Dumani         | Kunjuzwa       | Jean      | Greyling  |
| 9  | Janine         | Nel            | Jean      | Greyling  |
| 10 | Brenda         | Scholtz        | Jean      | Greyling  |
| 11 | Marinda        | Taljaard       | Jean      | Greyling  |

TEGOP\MSSQLSERVER01 (16.... | THUTEGOP\potsa (52) | University | 00:00:00 | 19 rows

3.

- a. `SELECT COUNT(mCode) AS NumberOf_Modules, SUM(mCredit) AS Total_Credits  
FROM Module  
WHERE mCode LIKE '%3%';` *will consider any module with a 3 anywhere.*

b.

| Results |                  | Messages      |
|---------|------------------|---------------|
|         | NumberOf_Modules | Total_Credits |
| 1       | 11               | 102           |

potsa (54) | University | 00:00:00 | 1 rows

4.

- a. `SELECT s.sNo, s.sFName, s.sLName, t.Mark as Highest_Mark, t.mCode as CourseCode  
FROM Student AS s JOIN Takes as t ON s.sNo = t.sNo  
JOIN Module AS m ON m.mCode = t.mCode  
WHERE m.mCode in ('WRPV302', 'WRWV202')  
and t.Mark=(  
select max(Mark)  
from Takes  
where mCode=t.mCode);` *not needed*

b.

Results

Messages

|   | sNo       | sFName    | sLName  | Highest_Mark | CourseCode |
|---|-----------|-----------|---------|--------------|------------|
| 1 | 218961694 | Busiswa   | Fisanti | 86           | WRPV302    |
| 2 | 220726494 | Andiswa   | Pillay  | 93           | WRWV202    |
| 3 | 218624687 | Siphumile | Mgilane | 93           | WRWV202    |

LSERVER01 (16....

THUTEGOP\potsa (54)

University

00:00:00

3 rows

5.

5.

- a. `SELECT m.mDescr, m.mCredit  
FROM Module AS m JOIN Teach as t ON m.mCode = t.mCode  
JOIN lecturer AS l ON l.lNo = t.lNo  
WHERE mCredit >= 9 AND l.lName IN ('Vogts', 'Nel');`

b.

| Results |                          | Messages |
|---------|--------------------------|----------|
|         | mDescr                   | mCredit  |
| 1       | Advanced programming 3.1 | 10       |
| 2       | Advanced Programming 3.2 | 11       |
| 3       | Project                  | 9        |

UTEGOP\potsa (53) | University | 00:00:00 | 3 rows

## Final version

## WRDV301 - ASSIGNMENT 3

31/31

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Mavovana, Kutloano |
| Surname, Name | Rasimeni, Sandile  |

1.

a.

```
select l1Name, l1FName, COUNT(Student.sSuperLNo) as NumOfStudents
from Lecturer join Department on lNo=HoDLno
join Student on lNo=sSuperLNo
group by l1Name, l1FName
having COUNT(Student.sSuperLNo)>1
```

b.

(b)

| Results |          | Messages |               |
|---------|----------|----------|---------------|
|         | l1Name   | l1FName  | NumOfStudents |
| 1       | Greyling | Jean     | 5             |
| 2       | Mbava    | Willard  | 2             |
| 3       | Venter   | Andre    | 2             |

GOP\potsa (54) | University | 00:00:00 | 3 rows

2.

a.

```
SELECT l1.FName as Lecturer_FName, l1.LName as Lecturer_LName, h1.FName
as HoD_FName, h1.LName as HoD_LName
FROM Lecturer as l1 INNER JOIN Department as d on l1.LName = d.dName
inner join Lecturer AS h on h.lNo=d.HoDLno
WHERE l1.lNo <> h.lNo
ORDER BY h.l1Name, l1.LName;
```

[8]

b.

|    | Lecturer_FName | Lecturer_LName | HoD_FName | HoD_LName |
|----|----------------|----------------|-----------|-----------|
| 1  | Eric           | Bashman        | Benita    | Barton    |
| 2  | Richard        | Betz           | Benita    | Barton    |
| 3  | Ernst          | Ferg           | Benita    | Barton    |
| 4  | Lynette        | Bamard         | Jean      | Greyling  |
| 5  | Dix            | Dixie          | Jean      | Greyling  |
| 6  | MC             | du Plessis     | Jean      | Greyling  |
| 7  | Ife            | Fashoro        | Jean      | Greyling  |
| 8  | Dumani         | Kunjuzwa       | Jean      | Greyling  |
| 9  | Janine         | Nel            | Jean      | Greyling  |
| 10 | Brenda         | Scholtz        | Jean      | Greyling  |
| 11 | Marinda        | Taljaard       | Jean      | Greyling  |

TEGOP\MSSQLSERVER01 (16.... | THUTEGOP\potsa (52) | University | 00:00:00 | 19 rows

3.

a.

```
SELECT COUNT(mCode) AS NumberOf_Modules, SUM(mCredit) AS Total_Credits
FROM Module
WHERE mCode LIKE '%3%';
```

[4]

b.

| Results |                  | Messages      |  |
|---------|------------------|---------------|--|
|         | NumberOf_Modules | Total_Credits |  |
| 1       | 11               | 102           |  |

potsa (54) | University | 00:00:00 | 1 rows

4.

a.

```
SELECT s.sNo, s.sFName, s.sLName, t.Mark as Highest_Mark, t.mCode as
CourseCode
FROM Student AS s JOIN Takes as t ON s.sNo = t.sNo
WHERE t.mCode in ( 'WRPV302', 'WRWV202' )
and t.Mark=(
select max(Mark)
from Takes
where mCode=t.mCode);
```

[9]

We removed the unnecessary  
line where we joined the Module  
table on m.mCode = t.mCode

b.

| Results |           | Messages  |         |              |            |
|---------|-----------|-----------|---------|--------------|------------|
|         | sNo       | sFName    | sLName  | Highest_Mark | CourseCode |
| 1       | 220726494 | Andiswa   | Pillay  | 93           | WRWV202    |
| 2       | 218624687 | Siphumile | Mgilane | 93           | WRWV202    |
| 3       | 218961694 | Busiswa   | Fisanti | 86           | WRPV302    |

ISSQLSERVER01 (16... | THUTEGOP\potsa (61) | University | 00:00:00 | 3 rows

5.

a.

```
SELECT m.mDescr, m.mCredit
FROM Module AS m JOIN Teach as t ON m.mCode = t.mCode
JOIN lecturer AS l ON l.lNo = t.lNo
WHERE mCredit >= 9 AND l.lName IN ('Vogts', 'Nel');
```

[4]

b.

| Results |                          | Messages |  |
|---------|--------------------------|----------|--|
|         | mDescr                   | mCredit  |  |
| 1       | Advanced programming 3.1 | 10       |  |
| 2       | Advanced Programming 3.2 | 11       |  |
| 3       | Project                  | 9        |  |

UTEGOP\potsa (53) | University | 00:00:00 | 3 rows

## ASSIGNMENT 4

Draft version

## Assignment 4-Draft

31  
35

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Mavovana, Kutloano |
| Surname, Name | Rasimeni, Sandile  |

1. [12]
- ER model is top down as we start with requirements then make entities then make relationships and its attributes. ✓
  - Normalization is the bottom-up process as it starts with information which is not organized and then packed into entities. ✓
2. [7]
- Option 1**  
There is no table for the Volunteer. The person entity is abstract and has no instance. ✓
  - Option 2**  
Only the super type is implemented, it will have nulls since friend does not receive salary. There are multiple null values since not all fields are applicable to everyone in the database, like an employee and a volunteer do not have a friend description. ✓
  - Option 3**  
Subtypes are not overlapping, and each person is one type only. It is already in 3NF ✓
- b. **Option 3 would be most appropriate** ✓  
Because it caters for every person, since you can still track the volunteer, employee and friends and will also not have null values since all the tables are implemented separately and it ensures good normalization, all the business rules are implemented as described. Volunteer can be recorded using Person since they only need the base data. It is already in 3NF, there are no partial or transitive dependencies on these tables. ✓

3. [14]
- BEGIN TRANSACTION;  
 UPDATE PRODUCT  
 SET PROD\_QoH = PROD\_QoH + 15  
 WHERE PROD\_CODE='ABC'; ✓  
 UPDATE COMPONENT  
 SET COMP\_QoH = COMP\_QoH - (45) ✓  
 WHERE COMP\_CODE = 'A'; ✓  
 UPDATE COMPONENT  
 SET COMP\_QoH = COMP\_QoH - (15) ✓  
 WHERE COMP\_CODE = 'B'; ✓  
 UPDATE COMPONENT  
 SET COMP\_QoH = COMP\_QoH - (30) ✓  
 WHERE COMP\_CODE = 'C'; ✓  
 COMMIT: ✓
  - | TRL ID | TRX NUM | PREV PTR | NXT PTR | OPERATION | TABLE         | ROW ID | ATTRIBUTE | BEFORE VALUE | AFTER VALUE |
|--------|---------|----------|---------|-----------|---------------|--------|-----------|--------------|-------------|
| 101    | 3       | NULL     | 102     | START     | *START TRANS* |        |           |              |             |
| 102    | 3       | 101      | 103     | UPDATE    | PRODUCT       | ABC    | PROD_QoH  | 2300         | 2315 ✓      |
| 103    | 3       | 102      | 104     | UPDATE    | COMPONENT     | A      | COMP_QoH  | 5600         | 5555 ✓      |
| 104    | 3       | 103      | 105     | UPDATE    | COMPONENT     | B      | COMP_QoH  | 1200         | 1185 ✓      |
| 105    | 3       | 104      | 106     | UPDATE    | COMPONENT     | C      | COMP_QoH  | 4500         | 4470 ✓      |
| 106    | 3       | 105      | NULL    | COMMIT    |               |        |           |              |             |

1 | Page



4. [6]
- The concurrency control problem that occurred is the lost updates. This happens when two transactions update the same data element and one of the updates is lost. ✓
    - Time 16 ✓
    - T2 reads C and updates it by (C: =C+A.) Then, T1 also updates C by using (C: =C+B), which fully ignores the update made by T2. Now since T1 never read T2's update version of C, it overwrites it. Hence T2's updates are lost. ✓
  - We can use Two-Phase locking (2PL) to ensure serializability. It defines how transactions release locks. Acquires all required locks without unlocking any data. ✓  
 We can use it in this way:  
 Time 9: ensure that T2 lock is locked when it tries to acquire a lock on C because T1 had already done it in Time 4. ✓  
 T2 must wait until T1 releases its lock on C at time 17 before it can read and write item C. ✓

*In what sequence would locks have to be obtained?*

Final version

31/05

## Assignment 4-FINAL

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Mavovana, Kutloano |
| Surname, Name | Rasimeni, Sandile  |

1.
  - a. ER model is top down as we start with requirements then make entities then make relationships and its attributes.
  - b. Normalization is the bottom-up process as it starts with information which is not organized and then packed into entities.
2.
  - a. **Option 1**  
There is no table for the Volunteer. The person entity is abstract and has no instance.
  - Option 2**  
Only the super type is implemented, it will have nulls since friend does not receive salary. There are multiple null values since not all fields are applicable to everyone in the database, like an employee and a volunteer do not have a friend description.
  - Option 3**  
Subtypes are not overlapping, and each person is one type only. It is already in 3NF
  - b. **Option 3 would be most appropriate**  
Because it caters for every person, since you can still track the volunteer, employee and friends and will also not have null values since all the tables are implemented separately and it ensures good normalization, all the business rules are implemented as described. Volunteer can be recorded using Person since they only need the base data. It is already in 3NF, there are no partial or transitive dependencies on these tables.
3.
  - a.
 

```
BEGIN TRANSACTION;
UPDATE PRODUCT
SET PROD_QoH = PROD_QoH + 15
WHERE PROD_CODE='ABC';
UPDATE COMPONENT
SET COMP_QoH = COMP_QoH - (45)
WHERE COMP_CODE = 'A';
UPDATE COMPONENT
SET COMP_QoH = COMP_QoH - (15)
WHERE COMP_CODE = 'B';
UPDATE COMPONENT
SET COMP_QoH = COMP_QoH - (30)
WHERE COMP_CODE = 'C' ;
COMMIT;
```

b.

| TRL ID | TRX NUM | PREV PTR | NXT PTR | OPERATION | TABLE         | ROW ID | ATTRIBUTE | BEFORE VALUE | AFTER VALUE |
|--------|---------|----------|---------|-----------|---------------|--------|-----------|--------------|-------------|
| 101    | 3       | NULL     | 102     | START     | *START TRANS* |        |           |              |             |
| 102    | 3       | 101      | 103     | UPDATE    | PRODUCT       | ABC    | PROD_QoH  | 2300         | 2315        |
| 103    | 3       | 102      | 104     | UPDATE    | COMPONENT     | A      | COMP_QoH  | 5600         | 5555        |
| 104    | 3       | 103      | 105     | UPDATE    | COMPONENT     | B      | COMP_QoH  | 1200         | 1185        |
| 105    | 3       | 104      | 106     | UPDATE    | COMPONENT     | C      | COMP_QoH  | 4500         | 4470        |
| 106    | 3       | 105      | NULL    | COMMIT    |               |        |           |              |             |

1 | Page

4.

a.

- (i) The concurrency control problem that occurred is the lost updates. This happens when two transactions update the same data element and one of the updates is lost.
- (ii) Time 16
- (iii) T2 reads C and updates it by  $(C := C + A)$ . Then, T1 also updates C by using  $(C := C + B)$ , which fully ignores the update made by T2. Now since T1 never read T2's update version of C, it overwrites it. Hence T2's updates are lost.

b.

We can use Two-Phase Locking (2PL) to ensure serializability. It defines how transactions release locks. Acquires all required locks without unlocking any data. We can use it in this way:

T1:

Acquires all required locks before accessing shared data:

Read\_Lock(C) before reading C (Time 4)

Performs computation:

 $C := C + B$ 

Writes back and then releases:

Write\_Item(C) (Time 16)

Unlock(C) – (Time 17)

T2:

Must wait until T1 has released its locks on C

Then, we can say T2 acquires Read\_Lock(C) before reading it (Time 9)

Does computation:  $C := C + A$ 

Write\_Item(C) (Time 12)

Releases locks: Unlock(C) – (Time 13)

What about B?

Which transaction started first, according to the set of operations given?

## ASSIGNMENT 5

Draft version

14  
19

### Assignment 5-Draft

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Mavovana, Kutloano |
| Surname, Name | Rasimeni, Sandile  |

1. a)

SELECT p.PatronID, p.PatronName, b.BookID, b.BookTitle, br.BranchID, brw.BorrowDate  
FROM PATRON as p INNER JOIN (BORROW as brw INNER JOIN BOOK as b ON brw.BookID = b.BookID)  
ON brw.PatronID = p.PatronID  
WHERE b.Genre = 'SCIFI' AND  
brw.BorrowDate = '2025-04-30' AND  
brw.ReturnDate IS NULL  
ORDER BY b.BorrowDate, b.ReturnDate;

The optimized query uses explicit JOINS in the FROM clause instead of the old old style Joins, which improves readability and maintainability which may make it easier to understand.

The reason for the use of Aliases is ensures that the attributes / column name are retrieved from the correct tables since two or more tables may have the same attribute name.

The equality conditions (e.g., b.Genre = 'SCIFI') is placed first since it is the most selective.

The date conditions (e.g., brw.BorrowDate = '2025-04-30') is placed next, as it can be selective but may also depend on the data distribution.

Null condition (e.g., brw.ReturnDate IS NULL) is placed lastly, as it can be less selective and may require additional processing.

b) Indexing could be used in the BorrowDate column, since it is the most selective attribute in the WHERE clause. It also has a High sparsity with no null values. Could be used on the ReturnDate, and it is the second selective column in the selection list under WHERE clause with High data sparsity (Which has high proportion of null values). *Missing a potential index*

2. a) The SUPPLIER database is Fragmented into SUPPLIER\_A and SUPPLIER\_B in Site A and C respectively, so this is a Distributed Request Operation.

b) Since the SUPPLIER and STOCK tables are fragmented in Site A and C in extent STOCK is also Distributed in Site B, So to support transcation at Site C (may have low current load) the database must minimally support the following operations:

**1. UPDATE:**

To modify existing rows in the SUPPLIER and STOCK table ensuring data is fully applied or not at all. Must ensure data atomicity and integrity of updates.

**2. INSERT:**

To add new rows to the ORDER and ORD\_LINE tables.

Ensures atomicity and integrity of inserts, including foreign key enforcement from the SUPPLIER and STOCK tables in this Site.

**3.Transaction Management:**

BEGIN TRANSACTION and COMMIT to manage the transaction as whole and incase of trash in the middle of processing or after.

Ensure ACID properties (Atomicity, Consistency, Isolation, Durability).

1 | Page

**4. Transaction Control:**

Since on Update, the query process will require ORD\_LINE and ORDER table, which are on other independent site B, then Site C will send a Remote Request to Site B.

*This is what is required for all operations.*

## Final version

17/19

## Assignment 5-Final

|               |                    |
|---------------|--------------------|
| Surname, Name | Potsane, Thutego   |
| Surname, Name | Mavovana, Kutloano |
| Surname, Name | Rasimeni, Sandile  |

1. a)
- ```
SELECT p.PatronID, p.PatronName, b.BookID, b.BookTitle, br.BranchID, brw.BorrowDate
FROM PATRON as p INNER JOIN (BORROW as brw INNER JOIN BOOK as b ON brw.BookID = b.BookID)
ON brw.PatronID = p.PatronID
WHERE b.Genre = 'SCIFI' AND
brw.BorrowDate = '2025-04-30' AND
brw.ReturnDate IS NULL
ORDER BY b.BorrowDate, b.ReturnDate;
```

The optimized query uses explicit JOINS in the FROM clause instead of the old old style Joins, which improves readability and maintainability which may make it easier to understand.  
The reason for the use of Aliases is ensures that the attributes / column name are retrieved from the correct tables since two or more tables may have the same attribute name.  
The equality conditions (e.g., b.Genre = 'SCIFI') is placed first since it is the most selective.  
The date conditions (e.g., brw.BorrowDate = '2025-04-30') is placed next, as it can be selective but may also depend on the data distribution.  
Null condition (e.g., brw.ReturnDate IS NULL) is placed lastly, as it can be less selective and may require additional processing.

- b)
- Indexing could be used in the BorrowDate column, since it is the most selective attribute in the WHERE clause. It also has a High sparsity with no null values.  
Could be used on the ReturnDate, and it is the second selective column in the selection list under WHERE clause with High data sparsity (Which has high proportion of null values).

Other potential index could be a Genre which is the last selective attribute in the WHERE clause. Which has a High sparsity with no null values (as there is low proportion of nulls).

2. a) The SUPPLIER database is Fragmented into SUPPLIER\_A and SUPPLIER\_B in Site A and C respectively, so this is a Distributed Request Operation.

b) Since the SUPPLIER and STOCK tables are fragmented in Site A and C in extent STOCK is also Distributed in Site B, So to support transaction at Site C (may have low current load) the database must minimally support the following operations:

#### 1. UPDATE:

The SUPPLIER is Distributed into two sites, and one Query Statement to access data from different Local sites which are Site A and Site C, then Transaction operation is a Distributed Request, to support the database.

The STOCK is distributed at Site A,b, and C, only single statement to access it, then the transaction operation required is Distributed Request on Update.

#### 2. INSERT:

The ORDER is located at Site B, and one Query statement to access it, then Site C will send a Remote Request to Site B.

The ORDER\_LINE is located at Site B, and one Query statement to access it, then Site C will send a Remote Request to Site B.

#### 3. Transaction Control:

The Transaction contains multiple query statements and remotely request access for data at Multiple sites, each remote request at a time.

## REFLECTION

### Most valuable concept:

The most valuable concept I've learned in this module is Distributed Database Systems (DDBS). Understanding DDBS has provided me with insights into how databases can be designed and managed across multiple sites through Communication networks, improving reliability, availability, and performance.

To which I was wondering on how businesses manage data in the same database more efficiently, hence now I have learnt that by Distributing Database into multiple sites, will be more efficient such a way that querying data will be faster than querying on undistributed database. 

Understanding DDBS has given me a solid foundation in designing and managing databases for complex systems. The concepts learned will be beneficial in my future projects, enabling me to create more efficient and scalable database systems.

Secondly, but lastly I've learned in this module that a good designed database will easier the Performance tuning, this highlights the importance of database design model to be well designed, and all activities associated with the DB design process have been valued accordingly, such as Determining the end-user views, outputs, and transaction requirements. Defining entities, attributes, domains and relationships then Draw ER diagram, normalize entity attributes. Validation of rules will reduce data anomalies on the Relation DB. Also when designing need to ensure entity and referential integrity and define all the user requirements, security is highly recommended with storage organization.

By all this, performance tuning will easily improve query performance, efficient database design handles high traffic and concurrent requests. Where faster data retrieval and processing minimize delays. So for future purposes, performance tuning enables organizations to make data-driven decisions, improve productivity, and enhance user experience.

### Aspects Done Well:

The lecture provided clear explanations of complex concepts, making it easier to understand. Having Quizzes, Class Activities, and Assignments helped me a lot, as we were also allowed to work in groups, so that we can help each other and discuss on what we understand, when we don't understand, we were also allowed to go and ask for help to our Lecture.

The strategy of giving assignment before the content is being thought in class, was very brilliant because to what you didn't understand when you were doing the draft assignment,

then you come to class and get to understand to the lecture and ask questions so that you can be able to re-do the assignment for the marked Draft assignment.

Providing lecture example, contributed a lot on understanding the content more practically. The hands-on experience with database management systems (DBMS) like SQL Server was very useful in reinforcing theoretical concepts. ✓

**Aspects Needing Improvement:**

Additional resources or sessions, if students can have extra session by the help of Demis (Student Assistant) or the Lecture per se, so that they will help us working on our assignments underlead, probably that will help students won't be triggered to go online (ChatGPT) to ask questions and copy-paste the assignment. Also, Students will gain more experience in applications of the content and advise for tests. *What additional sources would be useful?*

One life lesson I've taken away from this module is the importance of attention to detail in database design. A well-designed database can significantly impact the performance and scalability of an application. This attention to detail can be applied and consideration of life, where careful planning and consideration can lead to better outcomes. As the say, A good database results into a good performance. ✓